



E0 284: Digital VLSI Circuits
Laboratory Manual

Module 4:
Front-end ASIC Flow using Cadence Tools

Course Instructor:

Dr. Chetan Singh Thakur

Assistant Professor

Department of Electronic Systems Engineering
Indian Institute of Science, Bengaluru

Acknowledgement

This manual is adapted from a manual created by Cadence, titled “ASIC Design Flow”.

The content is updated where there are mismatches between the original document and the tool flow used in our laboratory.

This document is meant for educational purpose only.

We acknowledge the University Support Team, Cadence Design Systems, Bengaluru, who are the creators of the original document.

Contents

Acknowledgement.....	2
1. Getting started.....	4
Steps to Invoke Tools	4
Directory Structure	4
2. Verification	5
Opening NCLaunch	5
Compilation	7
Elaboration	8
Simulation.....	9
3. Synthesis	11
Synthesis without DFT.....	11
Synthesis with DFT (optional)	14
4. Logic Equivalence Checking	17
5. Timing Simulation	20

1. Getting started

Steps to Invoke Tools

- Log in to your linux account with the credentials shared.
 - Default location after login: /home/dvlsi_group*
- Enter the following commands (every time after login):
 - **cd worklib_user<x>/Digital_worklib**
 - **csh**
 - **source /fetch/cshrc**
- You would see the following message on the terminal:

```
*****  
***** Welcome to Cadence Tools *****  
*****
```

- Copy the database folder into your digital worklib (First time)
 - **cp ../../../../Shares/counter_database.zip .**
 - **unzip counter_database.zip**
 - **mv counter_database counter_database_12dec**
 - NOTE: Do all changes in this directory. DO NOT ALTER the original database.

Directory Structure

Enter the newly created directory using `cd <dir_name>` command. See all folders.

- **cd counter_database_12dec**
- **ls**

Let us understand the directory structure.

- Captable – Contains Cap table. Captable file contains resistance and capacitance values which are used to model the interconnect parasitics of a design.
- Constraints – Contains SDC file. Constraints file contains information regarding the clocks and I/Os of the design, such as clock period, input delay etc.
- Equivalence checking – Used to check logical equivalence between RTL and synthesized netlist.
lef – Contains library exchange files (lef). LEF file contains information for representing the physical layout of an integrated circuit in an ASCII format. It includes design rules and abstract information about the cells.
- lib – Contains library files for synthesis, with slow and fast versions of cells for setup and hold checks, respectively.
- physical design – folder to run the physical design of the synthesized netlist.
- rtl – Contains RTL design files (.v files)
- simulation – Contains testbench files. It is the run directory for simulations.
- synthesis – folder to run synthesis of the RTL.

2. Verification

IES (Incisive Enterprise Simulator) is the tool used for verification. Navigate to Simulation directory where you have kept your RTL and test bench (simulation directory).

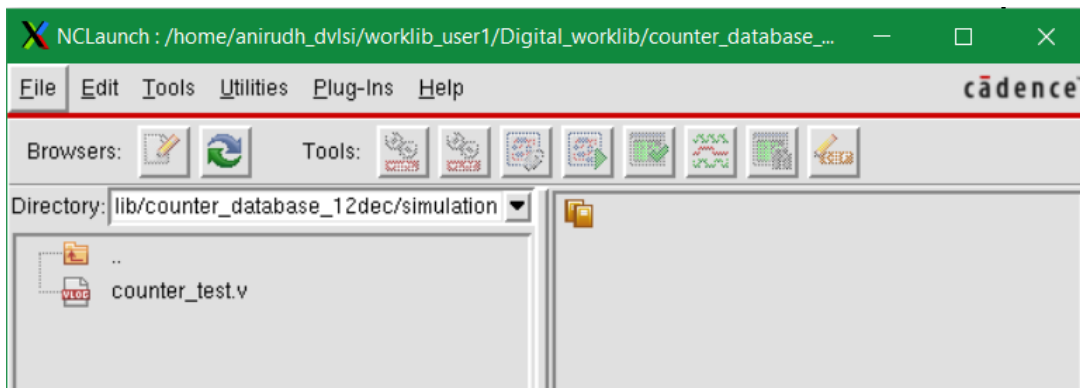
Opening NCLaunch

Run the following commands

cd simulation

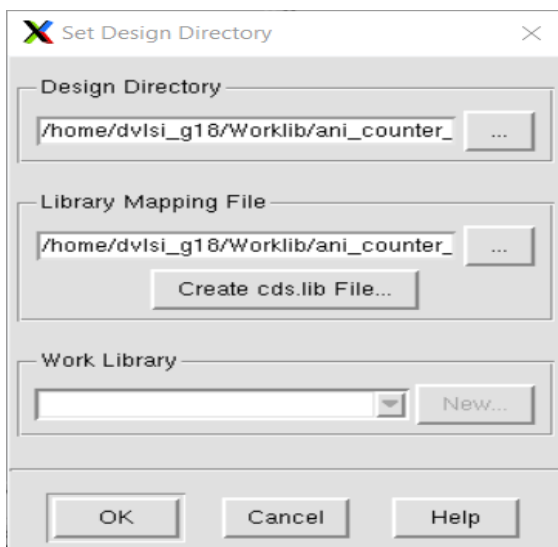
nclaunch &

The following NCLaunch window opens.

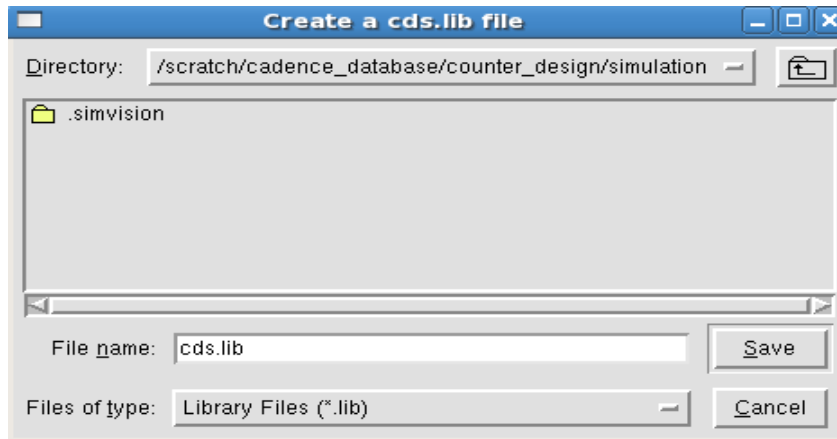


First time use in any folder - creation of cds.lib file

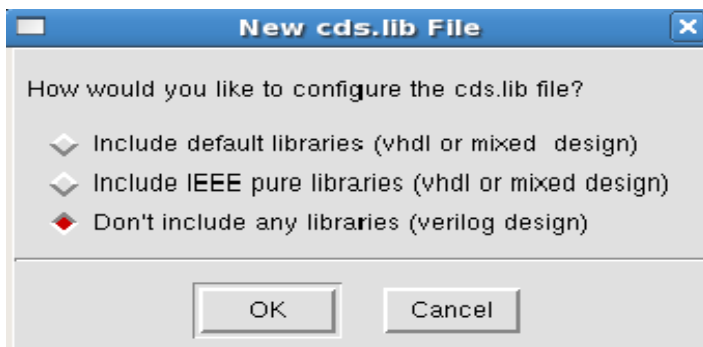
Click on File → Set Design Directory. The following window opens. Check that the design directory points to the folder you are in.



'Create cds.lib File' option and a 'Create a cds.lib file' window will open. Click 'Save' option.



A 'New cds.lib File' window will appear. Click any of the three options available depending on your RTL and click 'OK'. As the counter design is in verilog, the third option is selected.



Click OK to save these settings. This needs not be repeated as long as simulations are run in the same directory.

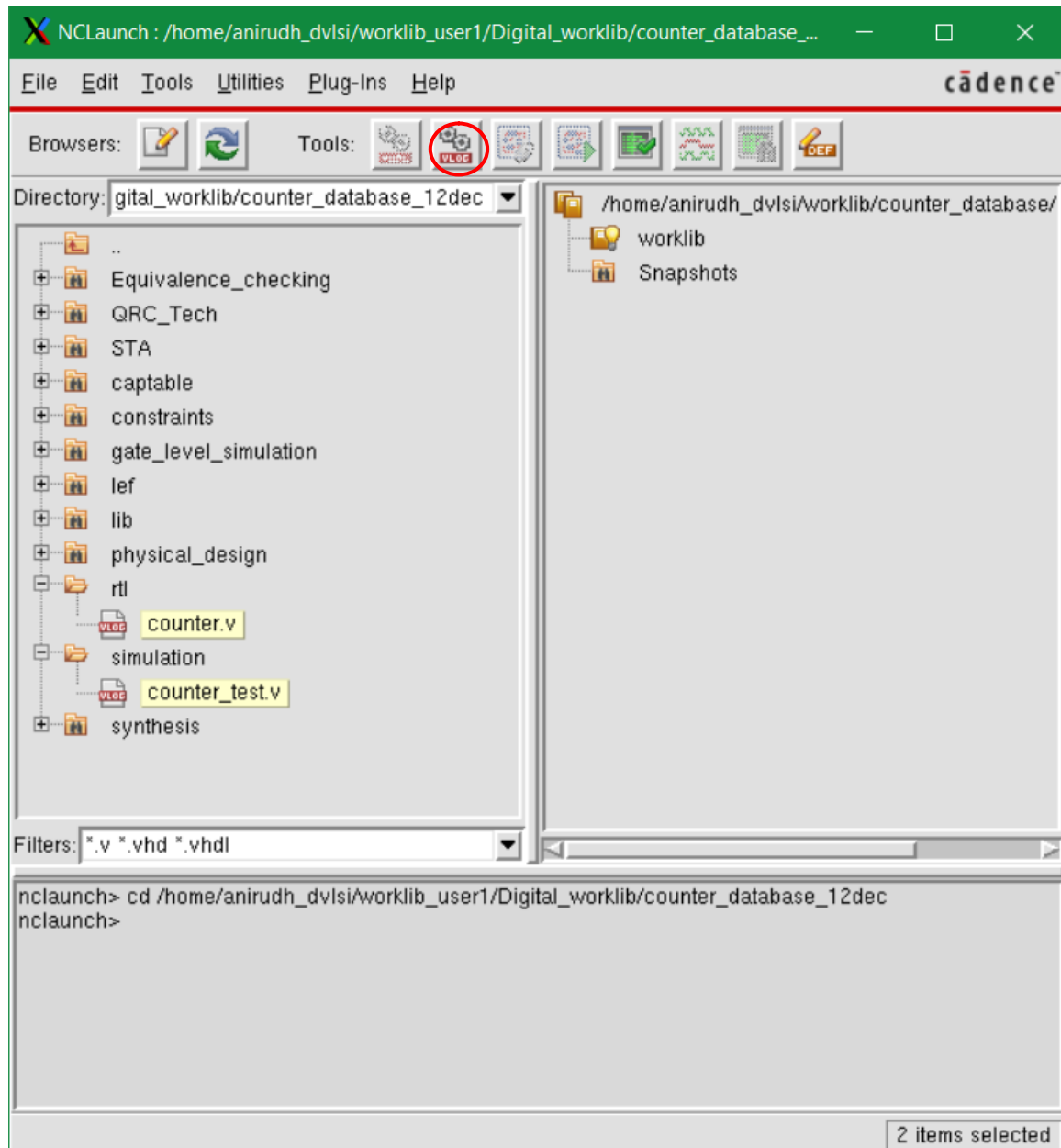
Compilation

- The next step is to compile (Checks syntax and semantics) the code. Navigate to one folder higher, using the  symbol.

Select the design files from RTL directory, and testbench from simulation directory. Ignore any design files in the simulation directory.

Choose the appropriate compilers.

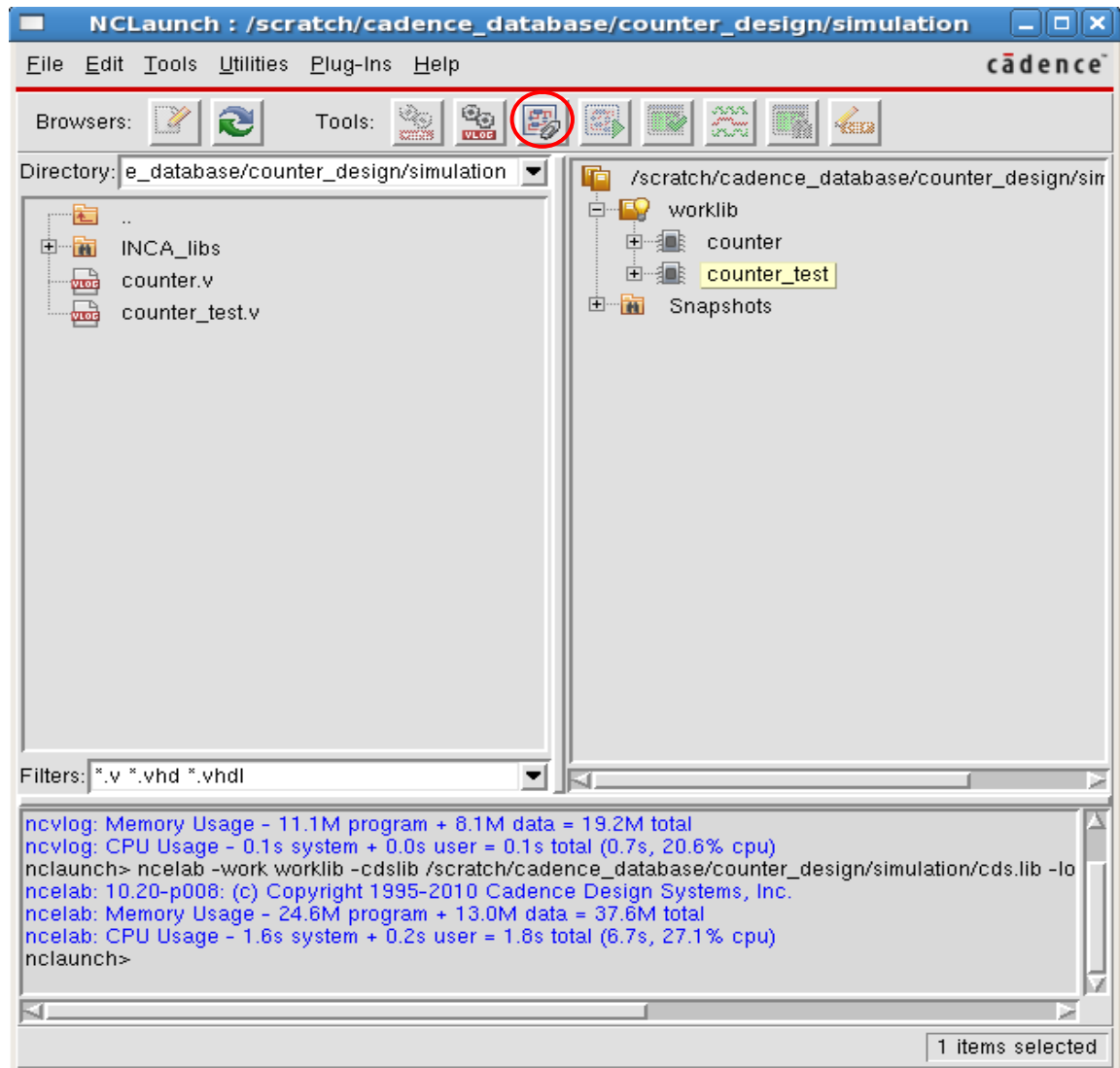
- *ncvlog* for Verilog designs. (choose ncvlog for counter design as the design is in verilog)
- *ncvhdl* for VHDL designs.



Any error in the code will be reported in the 'Command Entry Window'.

Elaboration

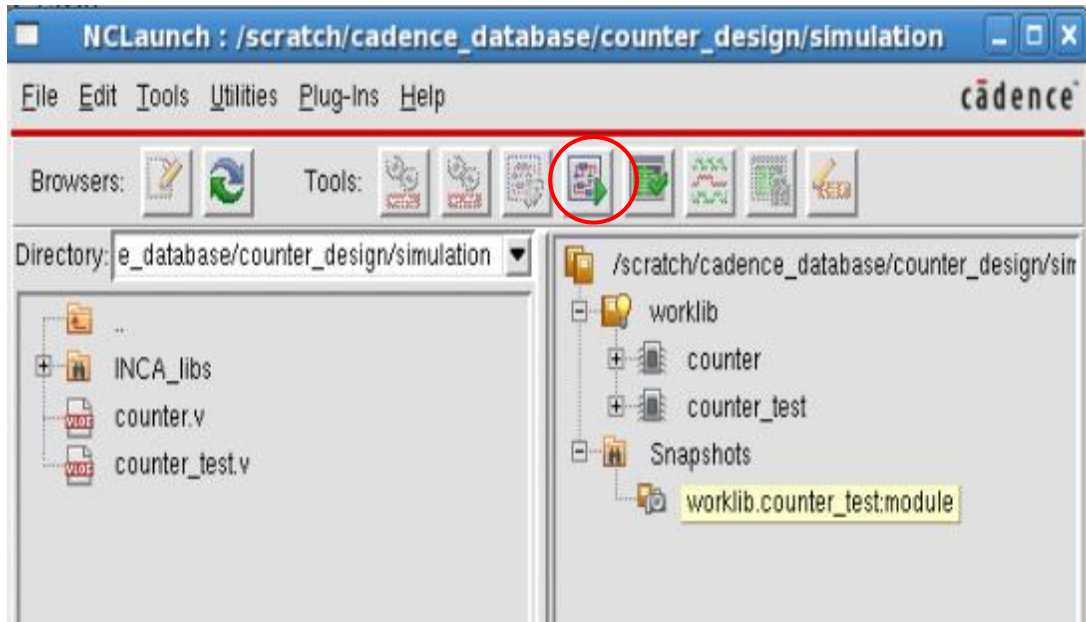
- After rectifying the errors in the code, the next step is elaboration (constructs design hierarchy and connects signals). Once the compilation is successfully completed, open the 'worklib' directory on the right side of the window and we can see the design objects created inside. Elaboration should be performed on the testbench as testbench is the top module at this stage and design is instantiated inside the testbench.
Select the testbench module (counter_test) and select the *launch elaborator*' (ncelab) key.



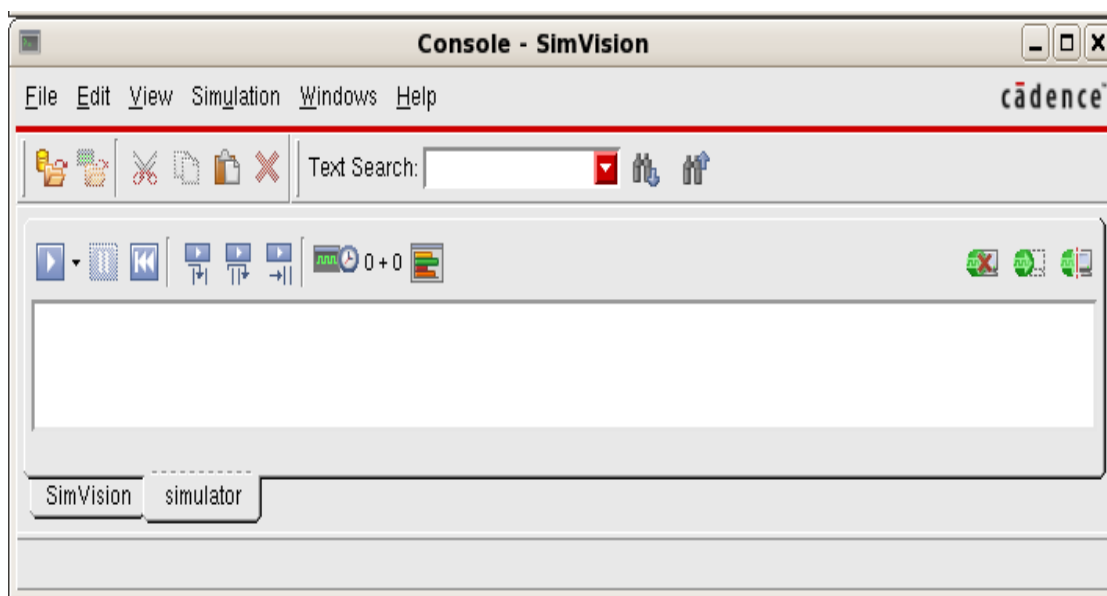
Simulation

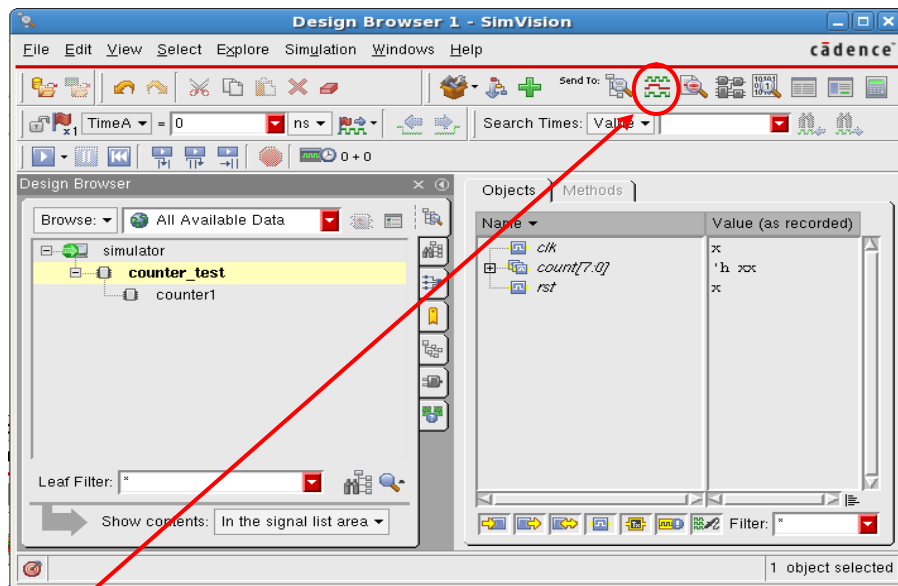
- After elaboration, the next step is simulation (executes simulation code). For this we have to send the snapshot generated during elaboration to the simulator.

Open the '*snapshots*' folder and select the snapshot and click on '*launch simulator*' option.

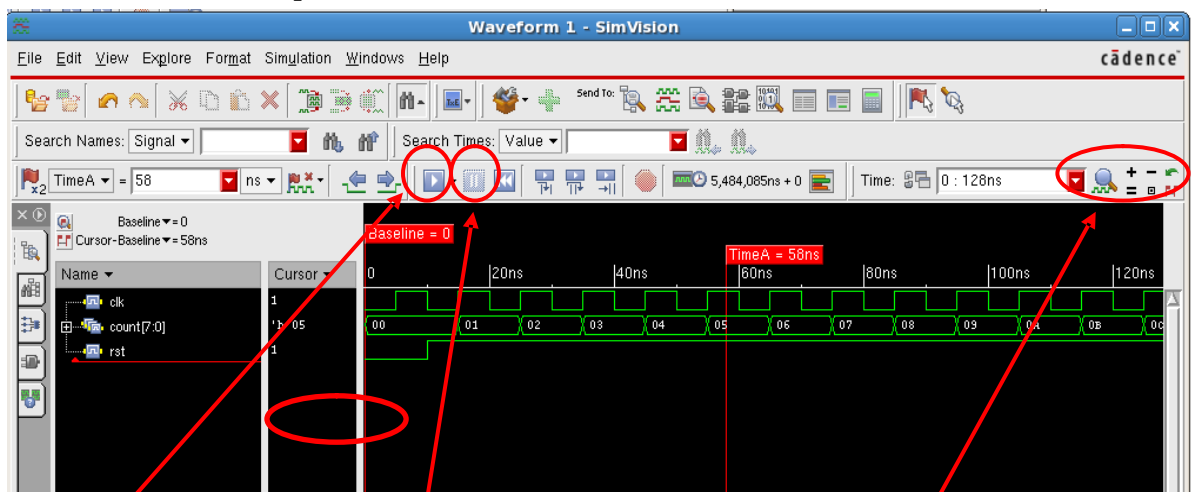


Launch simulator' will open 'Design Browser' and 'console' windows. '**Console -SimVision**' window can be used to perform simulation in command mode and hence can be minimized while using 'Design Browser -SimVision' window to run simulation in GUI mode.





- ☐ In the Design Browser window, select the testbench module (**counter_test**) and select the **'waveform'** option. A 'waveform -SimVison' window will appear
- ☐ In the waveform window, we can see different ports in the design. **Click on the Run simulation key to start the simulation.** Use the 'pause' key to interrupt or stop the simulation. Use different options like zoom in(+), zoom out(-) etc to analyze the plot. Use '=' symbol to see the full waveform plot.



Run Simulation

Interrupt Simulation

Zoom in, Zoom out etc..

- ☐ After verifying the design, close the tools. We can now proceed for synthesis.

To know more about tool options, use help or pdfs present inside the doc directory of the tool.

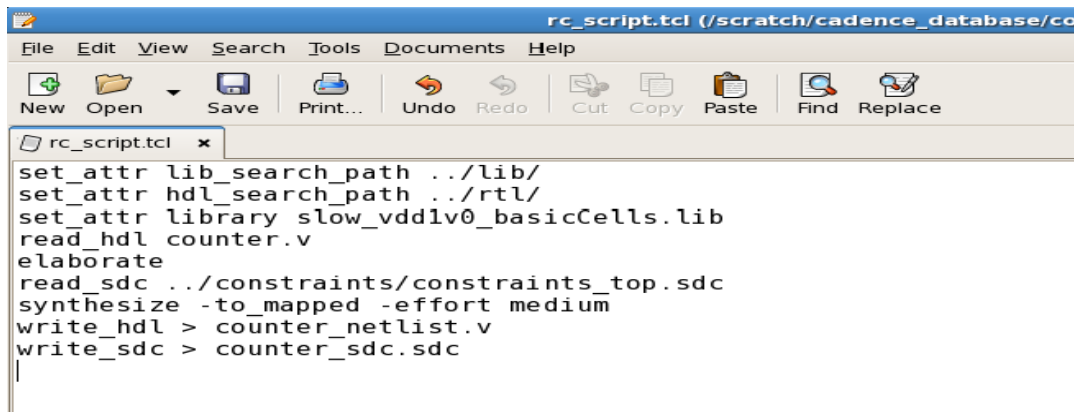
3. Synthesis

The tool used for synthesis (converting RTL to gate level netlist) is Genus.

Synthesis without DFT

Synthesis Script Description

We use `rc_script.tcl` to run the synthesis.



```
rc_script.tcl (/scratch/cadence_database/co
File Edit View Search Tools Documents Help
New Open Save Print... Undo Redo Cut Copy Paste Find Replace
rc_script.tcl x
set_attr lib_search_path ../lib/
set_attr hdl_search_path ../rtl/
set_attr library slow_vddlv0_basicCells.lib
read_hdl counter.v
elaborate
read_sdc ../constraints/constraints_top.sdc
synthesize -to_mapped -effort medium
write_hdl > counter_netlist.v
write_sdc > counter_sdc.sdc
```

- The necessary inputs to perform synthesis are RTL, standard cell library and constraints.
- Let us see the usage and purpose of each command.

- `set_attribute lib_search_path <library path>`

This command will set the path for the standard cell library.

- `set_attribute hdl_search_path <rtl path>`

This command will set the path for rtl files.

- `set_attribute library <library name>`

This command will read the specified standard cell library from the specified library path.

- `read_hdl <rtl design>`

This command will read the rtl design.

Note: - If the design is hierarchical or has multiple modules instantiated inside the top module, use curly braces '{ }' to mention all modules including the top design.

E.g.:- `read_hdl {top.v sub1.v sub2.v}`

Here `top.v` is the top module and `sub1.v` and `sub2.v` are the sub modules that are instantiated inside the top module.

- `elaborate`

The elaborate command constructs design hierarchy and connects signals.

- **read_sdc < sdc file name with path>**

This command reads in the timing constraints file. Here we have to provide the constraints file name along with the path. Explanation on constraints file is provided

- **Synthesize -to_mapped -effort medium**

This command will perform synthesis by combining the generic, mapped and incremental synthesis and - effort medium command specifies the synthesis effort. The effort can be set to 'low', 'medium' or 'high' depending upon the scenario.

The following commands write the outputs of synthesis - the netlist, the constraints file, and the delay file for basic gates.

write_hdl > counter_netlist.v

write_sdc > counter_sdc.sdc

write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge > delays.sdf

- Include all the above commands in the script file.

Note: - In counter design, you can see a script file 'rc_script.tcl' inside the synthesis directory. Please open the script file for further understanding.

Timing Constraints or SDC file description

- Using SDC, we define clock period, pulse width, rise and fall time, uncertainty and also input and output delays for different signals. Below is the constraints file used in counter design.

```
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.1 [get_ports "clk"]
set_input_delay -max 1.0 [get_ports "rst"] -clock [get_clocks "clk"]
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]
```

- **Create_clock -name -period 10 -waveform {0 5} {get_port "clk"}.**

This command will define clock with period 10ns and 50% duty cycle and signal is high in the first half.

- 'Set_clock_transition -rise/fall' command defines the transition delay for clock.
- 'Set_clock_uncertainty' command will set the uncertainty due to (clock skew and jitter).
- 'Set_input/output_delay' command will specify the input and output delay used for

timing slack calculations.

- Keep the constraints file inside the constraints directory.

Running Synthesis

- Use the below command to invoke RTL compiler along with the script file.

genus -legacy_ui -f <script file name with path>

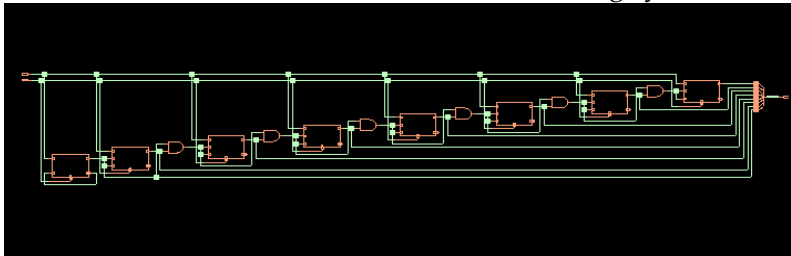
In case of the counter design, the command will be

genus -legacy_ui -f rc_script.tcl

The tool will execute each command mentioned inside the script file one by one.

Note: - If the script file is in the current working directory (synthesis directory), we need not have to provide the path for the script.

Ensure that there are no errors in the terminal during synthesis.



report qor command added in the script file will show the highlights of the synthesis report.

After synthesis, to see the schematic, launch the gui using the below command.

'**gui_show**' in the terminal. Click on the Plus icon and choose schematic.

Use '**gui_hide**' command to close the gui.

To view the synthesis reports, enter the following commands in the terminal:

report timing

report area

report power

Note: - Use just report command to know what all different reports you can dump out from genus.

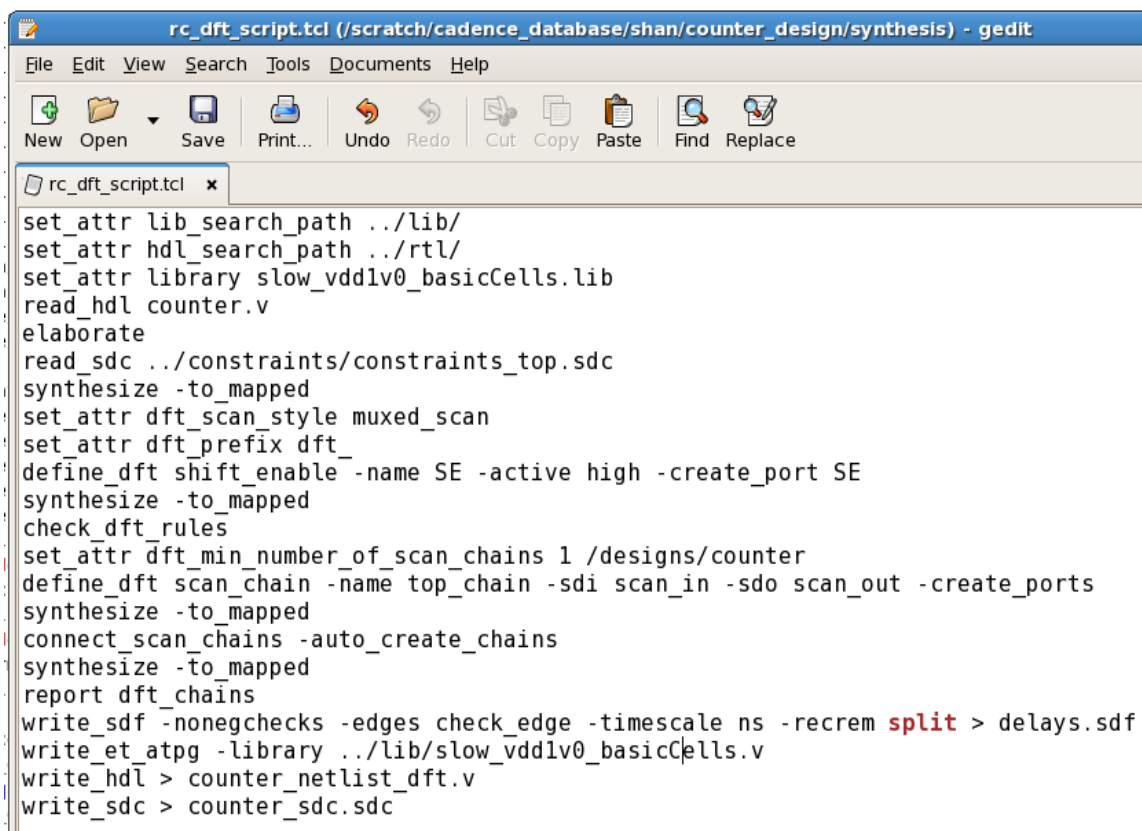
- After completing synthesis, exit the tool using the '**exit**' command in the terminal.

Synthesis with DFT (optional)

- Change directory to synthesis and locate the rc_dft_script.tcl.
- Now let us look at the content of the run script(rc_dft_script.tcl). Here is a breakdown of the script

flow for clarity:

- Load all the design files and elaborate
- Read sdc (in constraints dir)
- Read in DFT setup
- Synthesize to GENERIC
- Synthesize to MAPPED
- Run DFT flow
- Synthesis to MAPPED
- Write results and database
- Write ET files and ATPG flow



```
rc_dft_script.tcl (/scratch/cadence_database/shan/counter_design/synthesis) - gedit
File Edit View Search Tools Documents Help
New Open Save Print... Undo Redo Cut Copy Paste Find Replace
rc_dft_script.tcl x
set_attr lib_search_path ../lib/
set_attr hdl_search_path ../rtl/
set_attr library slow_vddl0_basicCells.lib
read_hdl counter.v
elaborate
read_sdc ../constraints/constraints_top.sdc
synthesize -to_mapped
set_attr dft_scan_style muxed_scan
set_attr dft_prefix dft
define_dft shift_enable -name SE -active high -create_port SE
synthesize -to_mapped
check_dft_rules
set_attr dft_min_number_of_scan_chains 1 /designs/counter
define_dft scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports
synthesize -to_mapped
connect_scan_chains -auto_create_chains
synthesize -to_mapped
report dft_chains
write_sdf -nonegchecks -edges check_edge -timescale ns -recrem split > delays.sdf
write_et_atpg -library ../lib/slow_vddl0_basicCells.v
write_hdl > counter_netlist_dft.v
write_sdc > counter_sdc.sdc
```

Let us understand the DFT specific commands in rc_dft_script.tcl file.

- The DFT scan FF style for scan replacement using the below command.

```
set_attr dft_scan_style muxed_scan
```

- Prefix is added to name of DFT logic that is inserted using the below command.

```
set_attribute dft_prefix DFT_/_
```

- Define the test signals (define_dft shift_enable) using the below command.

```
define_dft shift_enable -name {scan_en} -active {high} -create_port {scan_en}
```

- It is recommended you check DFT rules multiple times during a DFT flow using the below command
check_dft_rules

```
Summary of check_dft_rules
*****
Number of usable scan cells: 48
Clock Rule Violations:
-----
    Internally driven clock net: 0
    Tied constant clock net: 0
    Undriven clock net: 0
    Conflicting async & clock net: 0
    Misc. clock net: 0

Async. set/reset Rule Violations:
-----
    Internally driven async net: 0
    Tied active async net: 0
    Undriven async net: 0
    Misc. async net: 0

Total number of DFT violations: 0

Total number of Test Clock Domains: 1
Number of user specified non-Scan registers: 0
Number of registers that fail DFT rules: 0
Number of registers that pass DFT rules: 8
Percentage of total registers that are scannable: 100%
```

As you can see that there are no registers that fail DFT rules, which means that all of 8 registers are eligible for scan connection.

- Specify the number of scan chains required to connect all FF's using the below command. Here we have used 1 scan chain.

```
set_attr dft_min_number_of_scan_chains 1 /designs/counter
```

- Specify the scan in and scan out ports of the scan chain using the command

```
define_dft scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports
```

- Once scan ports has been created, perform the technology depended synthesis using the below command.

```
synthesis -to_mapped.
```

- Now connect the Scan chains using connect_scan_chains RC command. This will include all original FF's that were mapped to scan flops.

```
connect_scan_chains -auto_create_chains
```

- You can view the dft chains using the below command as shown.

```
report dft_chains
```

```

Reporting 1 scan chain (muxed_scan)

Chain 1: top_chain
scan_in:      scan_in
scan_out:     scan_out
shift_enable: SE (active high)
clock_domain: clk (edge: rise)
length: 8
  bit 1 count_reg[0] <clk (rise)>
  bit 2 count_reg[1] <clk (rise)>
  bit 3 count_reg[2] <clk (rise)>
  bit 4 count_reg[3] <clk (rise)>
  bit 5 count_reg[4] <clk (rise)>
  bit 6 count_reg[5] <clk (rise)>
  bit 7 count_reg[6] <clk (rise)>
  bit 8 count_reg[7] <clk (rise)>
-----

```

We will now run the final ATPG analysis and vector generation. This step will take the final scan chains and run through the ET flow for basic ATPG. This flow is implemented by the command

```
write_et_atpg -library ./Lib/slow_vdd1v0_basiccells.v
```

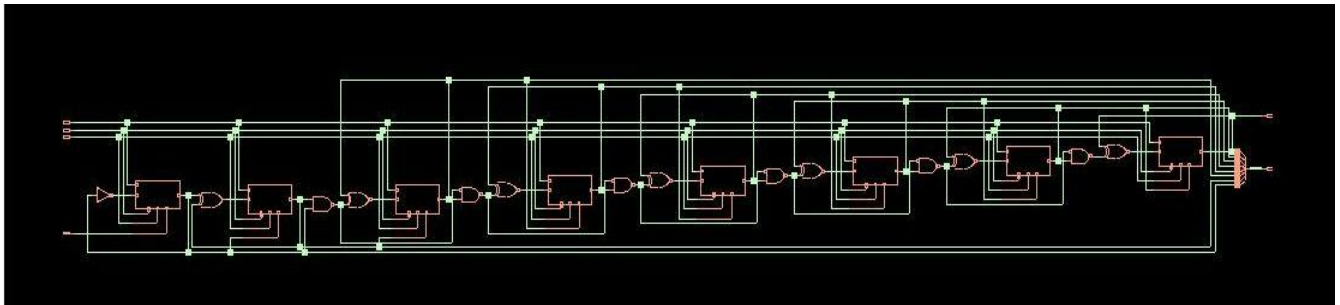
It will generate a directory '**et_scripts**' in current working location.

- ☐ Write out the final netlist, SDF & constraints using the below commands.

```
Write_hdl > counter_netlist.v
```

```
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge > delays.sdf
```

```
Write_sdc > count_sdc.sdc
```



Change directory to **et_scripts** to see the files that are generated by Genus.

- counter.et_netlist.v (completed verilog netlist used for ET)
- runet.atpg (ET ATPG run script)
- counter.FULLSCAN.pinassign (ET file specifying IO test behavior)
- et_check.sh (self error checking file used by runet.atpg)
- run_fullscan_sim (NC-sim script to verify ATPG patterns)

4. Logic Equivalence Checking

Conformal LEC is a tool used for formal verification of designs at various stages in the flow. Formal verification is the process of verifying designs using mathematical methods. Equivalence Checking is the process of verifying the correctness of a modified or transformed design (revised design) by comparing it with it with a reference design (golden design).

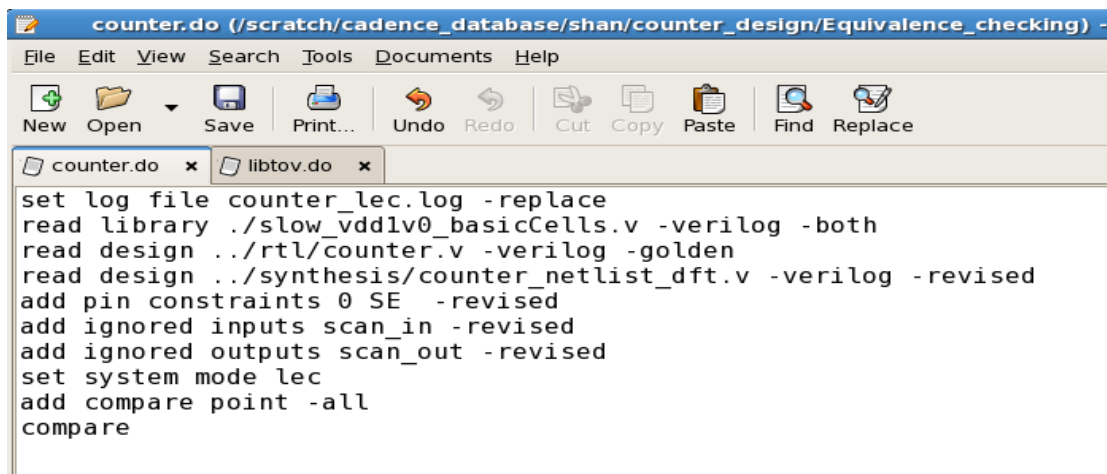
- ❑ **Enter the Equivalence_checking directory.**

- ❑ Invoke Conformal LEC inside Equivalence checking directory in non-GUI by using the **command**
lec -xl -nogui -color -64 -dofile counter.do

- ❑ **-xl** :- Launches Encounter® Conformal® LEC
- ❑ **-nogui** :- Starts the session in non-GUI mode
- ❑ **-color** :-Turn on color-coded messaging when in non-GUI mode
- ❑ **-64** :- Runs the Encounter® Conformal® software in 64-bit mode
- ❑ **-dofile <filename>** :- Runs the script <filename> after starting LEC

If the LEC Result is a PASS, exit the tool with the 'exit' command, confirm with 'yes'.

DOFILE



```
set log file counter Lec.log -replace
read library ./slow_vddlv0_basicCells.v -verilog -both
read design ../rtl/Counter.v -verilog -golden
read design ../synthesis/counter_netlist_dft.v -verilog -revised
add pin constraints 0 SE -revised
add ignored inputs scan_in -revised
add ignored outputs scan_out -revised
set system mode lec
add compare point -all
compare
```

Let us understand the content of the dofile.

Dofile is a script file used to run LEC and below is an example.

Below are the basic set of commands used.

- ❑ Save log file.

- set log file <filename.log> -replace**

- Save log file and replaces if any log file exist with same name if any.

- ❑ Read the Verilog library by entering:

- read library <filename> -verilog -both**

[Both verilog and liberty format can be used but verilog format is preferred. Steps to generate .v from .lib using Conformal is mentioned at the end of this session]

- ☐ -verilog :- to indicate that library is in Verilog format
- ☐ -both :- use same library to model or structure both golden and revised design.

- ☐ Read the Golden Design (RTL)
read design <filename> -verilog -golden
 - ☐ -verilog :- to indicate that RTL is coded in Verilog
 - ☐ -golden :- to input the golden design

- ☐ Read the Revised Design:
read design <filename> -verilog -revised
 - ☐ -verilog :- to indicate that netlist is in Verilog
 - ☐ -revised :- to input the revised design

- ☐ Ignore the scan input(scan_in) and Scan output (scan_out) pins (as these instances are not available in golden design and primary output key point is compare point)
add ignored inputs scan_in -revised [ignores scan_in pin]
add ignored outputs scan_out -revised [ignores scan_out pin]
- ☐ Constraint the scan enable (SE) pin to zero to keep the revised design in functional mode.
add pin constraints 0 SE -revised [tool keeps the design in functional mode and ignore scan_in pin while compare. Also scan_in is not a compare point]
Set system mode lec

Note: Conformal lec got two modes of operation i.e. SETUP mode and LEC mode. Setup mode is used to prepare the design to be compared. Any command that affects the way the design is modeled will need to be issued in this mode. LEC mode is where the designs will get modeled, key points mapped and where the compare process takes place.

- ☐ Compare golden Vs. revised netlist
add compare points -all
compare
report verification

Once the compare process is completed, Conformal LEC will print a summary report that tells how many key points are equivalent, non-equivalent, aborted and not compared. It will also report a table of all violated checklist items with PASS/FAIL status.

```
-----  
// Command: add compare point -all  
// 16 compared points added to compare list  
// Command: compare
```

```
=====
```

Compared points	P0	DFF	Total
Equivalent	8	8	16

```
=====
```

Note: Use command 'set gui on' to turn on GUI window. In case of mapping issue or comparison issue or not equivalence, use mapping manager or debug manager or Schematic viewer options in LEC to resolve the issue.

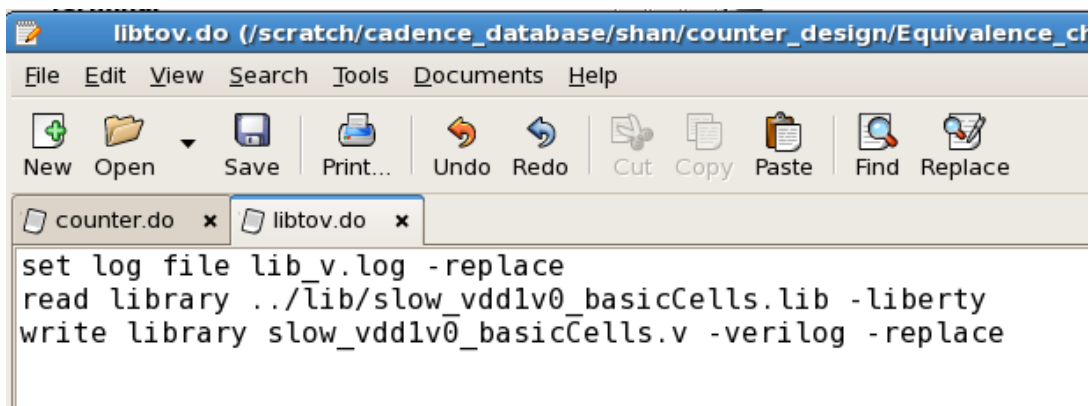
Note: Same is the flow to compare netlist generated at different stages of physical design. Use proper modelling directives and constraints.

Creating .v from .lib (This is done internally by counter.do file)

- ☐ Read library in liberty(.lib) format by using the command "read library <.lib file> -liberty -both"
- ☐ Write out the verilog file by using the command "Write library <file name*>-verilog"

*file name can be any name with extension .v

Below is a example dofile to generate '.v' from '.lib'



```
libtov.do (/scratch/cadence_database/shan/counter_design/Equivalence_ch...
File Edit View Search Tools Documents Help
New Open Save Print... Undo Redo Cut Copy Paste Find Replace
counter.do x libtov.do x
set log file lib_v.log -replace
read library ../lib/slow_vddlv0_basicCells.lib -liberty
write library slow_vddlv0_basicCells.v -verilog -replace
```

5. Timing Simulation

The pre-synthesis simulation assumed all gates to possess zero delay.

After synthesis, we get the netlist and the sdf file that contains the delay information of each gate. So, we need to simulate the netlist with these delays, and ensure that the design works in presence of these component delays.

Navigate to the gate_level_simulation folder. This folder contains the slow cells library file to check that the design works even with worst case delays, and the testbench.

Copy the SDF file from the synthesis folder to this folder using the command:
cp ../synthesis/delays.sdf .

Open nclaunch using the command:
nclaunch &

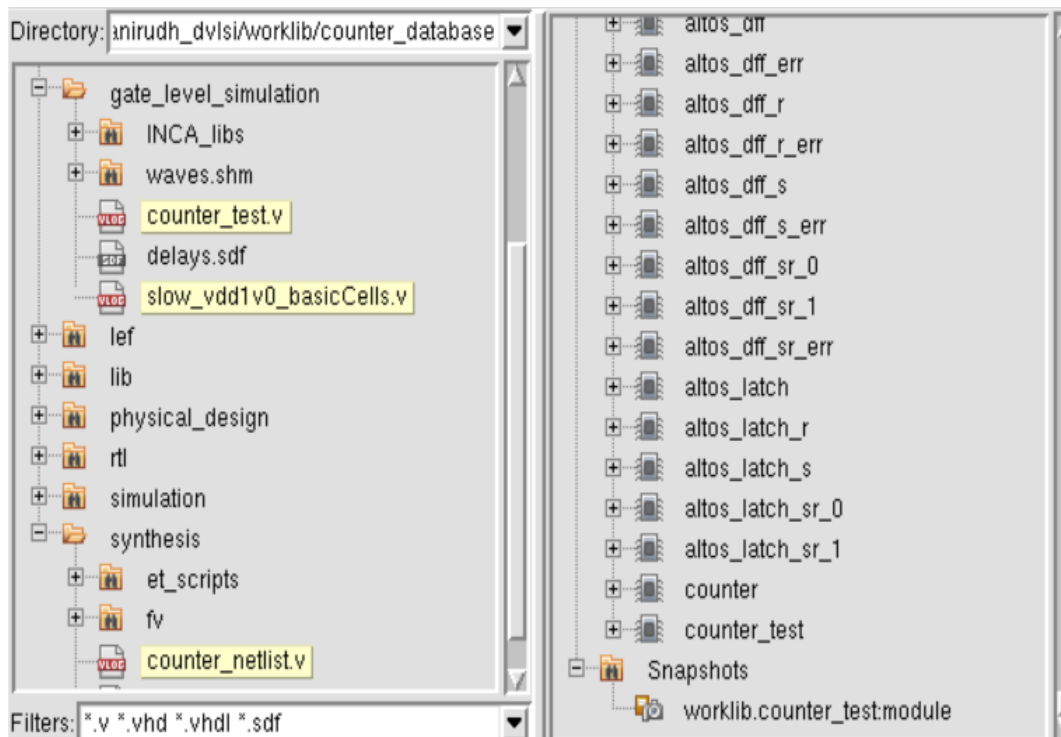
Open the testbench file. You can observe the following initial block.

```
initial
begin
  $sdf_annotate("delays.sdf", counter_test.counter1, "sdf.log", "MAXIMUM");
```

This block takes the delays.sdf file and back-annotates the delays in the netlist, taking the maximum delay into account. The hierarchical path to the top module of design must also be given. The log file will be stored in sdf.log.

The simulation process is the same as mentioned in the verification section. Open NCLaunch, create a new cds.lib file in this directory, when simulation is done here for the first time.

In the NCLaunch browser, navigate to the netlist file in the synthesis folder, and click on it. Also select (using CTRL key) the testbench and slow cells library file, and compile all of them

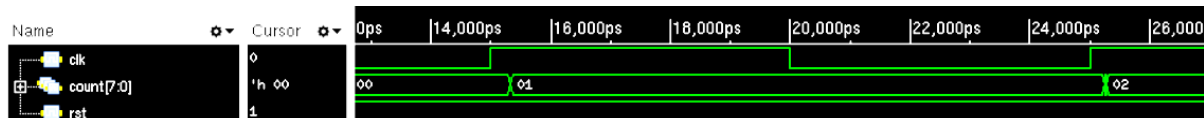


simultaneously. Once the design is compiled, you will see standard gates and logic blocks that are used in the design.

On elaborating the testbench, in the console window, you must see a message “Reading SDF File ...” with no errors. Annotated = 100% must be seen.

```
nclaunch> ncelab -work worklib -cdslib /home/anirudh_dvlsi/worklib/counter_database/gate_le
ncelab(64): 15.20-s077: (c) Copyright 1995-2019 Cadence Design Systems, Inc.
ncelab: *W,FILDNE: File pathname '/home/anirudh_dvlsi/worklib/counter_database/gate_level_
Reading SDF file from location "delays.sdf"
ncsdfc: Memory Usage - 18.8M program + 12.4M data = 31.2M total
ncsdfc: CPU Usage - 0.0s system + 0.0s user = 0.0s total (0.0s, 100.0% cpu)
SDF statistics: No. of Pathdelays = 52 Annotated = 100.00% -- No. of Tchecks = 64
ncelab: Memory Usage - 49.9M program + 33.8M data = 83.8M total (Peak 88.1M)
ncelab: CPU Usage - 0.1s system + 0.1s user = 0.1s total (0.1s, 88.4% cpu)
ncelab: ANNOTATION Time - 0.0s
```

Run the simulation and observe the waveform in SimVision to check functionality. Observe that the registers no longer change immediately at clock edge, but take some time to change.



You may also encounter glitches in multi-bit signals. But these can be ignored unless they happen very close to the next clock edge. Here the counter changes as $1 \rightarrow 0 \rightarrow 2$. Expected behaviour is $1 \rightarrow 2$.

